ASSIGNMENT 4 – CO4

DESIGNING REUSABLE CLASS HIERARCHIES

DSA0106-C++

Student Name: R. Peeran Basha

Register Number: 192472244

Department: Computer Science and Engineering

Institution: SIMATS Engineering – Saveetha School of Engineering

TASK 1 – CLASS DESIGN REVIEW

Task 1.1 – Diagnose the Design

The original payroll classes have the following design deficiencies:

- Repeated fields: Professor and LabAssistant repeat name, id and basic salary information.
- Repeated pay rule: Professor and LabAssistant both calculate the 42% allowance independently.
- Public data members: Any part of the program can directly change names, IDs and salary values, so invalid data can be stored.
- Inconsistent names: name/id are called nm/code in GuestFaculty, making common processing difficult.
- Unused field: da is declared in Professor and LabAssistant but is not used by pay().
- Magic number: 0.42 is embedded in the pay formula. A named constant is clearer and easier to maintain.
- Poor extensibility: adding another staff category requires another separate class and repeated logic.
- No common polymorphic interface: the three staff types cannot be processed uniformly through one base-class pointer or reference.

Task 1.2 – Repair the Design

Create an abstract Staff base class for common information. Keep common data protected (or private with accessors) and make pay() a pure virtual function. Each staff category overrides pay(). The allowance percentage is stored as a named constant.

```
#include <iostream>
#include <string>
#include <vector>
#include <memory>
#include <iomanip>

constexpr double DA_RATE = 0.42;

class Staff {
protected:
    std::string name;
```

```
    int id;
public:
    Staff(const std::string& n, int i) : name(n), id(i) {}
    virtual double pay() const = 0;
    virtual void display() const {
        std::cout << id << " " << name << " Pay: "
                  << std::fixed << std::setprecision(2) << pay() << '\n';
    }
    virtual ~Staff() = default;
};

class Professor : public Staff {
    double basic, research;
public:
    Professor(const std::string& n, int i, double b, double r)
        : Staff(n, i), basic(b), research(r) {}
    double pay() const override {
        return basic + basic * DA_RATE + research;
    }
};

class LabAssistant : public Staff {
    double basic;
public:
    LabAssistant(const std::string& n, int i, double b)
        : Staff(n, i), basic(b) {}
    double pay() const override {
        return basic + basic * DA_RATE;
    }
};

class GuestFaculty : public Staff {
```

```
    double hours, ratePerHour;
public:
    GuestFaculty(const std::string& n, int i, double h, double r)
        : Staff(n, i), hours(h), ratePerHour(r) {}
    double pay() const override {
        return hours * ratePerHour;
    }
};

int main() {
    std::vector<std::unique_ptr<Staff>> staff;
    staff.push_back(std::make_unique<Professor>("Anita", 101, 50000, 5000));
    staff.push_back(std::make_unique<LabAssistant>("Ravi", 102, 30000));
    staff.push_back(std::make_unique<GuestFaculty>("Meena", 103, 20, 800));

    for (const auto& person : staff)
        person->display();
}
```

UML-style class diagram (text representation):

```
              Staff
        --------------------------
        # name : string
        # id   : int
        --------------------------
        + pay() : double {abstract}
        + display() : void
        + virtual destructor
          /         |         \
         /          |          \
  Professor     LabAssistant   GuestFaculty
  ---------     ------------   ------------
  - basic       - basic        - hours
```

- research - ratePerHour

+ pay() + pay() + pay()

Sample expected output:

101 Anita Pay: 76000.00

102 Ravi Pay: 42600.00

103 Meena Pay: 16000.00

Task 1.3 – Design Defence (within 100 words)

The Staff base class stores information common to every staff member and defines one common pay() interface. Professor, LabAssistant and GuestFaculty provide their own pay() implementations. The payroll program can therefore process every staff object through a Staff pointer or reference without checking its type. To add a new category, a developer creates one new derived class and overrides pay(). Existing classes and the payroll loop do not need to be changed. This reduces duplication and makes the design easier to extend.

Alternative Concept and Complexity Analysis

With one Staff class, a type field could identify Professor, LabAssistant or GuestFaculty. The pay() method would use a switch statement and store fields for every possible category. Each pay() call is O(1) in both designs because it performs a fixed amount of work. A single-class object may waste space because fields for unrelated categories remain unused. With inheritance, each object stores only the fields needed by its actual category, plus normal object/polymorphism overhead. When a new category is added, the single-class design usually changes the type definition, pay() switch and possibly constructors/display logic. In the hierarchy, a new derived class can be added without changing existing classes or the common loop.

TASK 2 – FROM REQUIREMENT TO HIERARCHY

Task 2.1 – Identify the Relationships

Person is the common base because every student and staff member is a person. Student is-a Person and Staff is-a Person, so both use public inheritance. TeachingAssistant is-a Student and is-a Staff, so it uses multiple inheritance. ResearchScholar is-a Student, so it uses multilevel inheritance. A TeachingAssistant contains a Course or other supporting object only if the requirement says it owns that object; this is a has-a relationship implemented through composition.

Relationship	**Type**	**Reason**
Student → Person	Single inheritance	A student is a person.
Staff → Person	Single inheritance	A staff member is a person.
TeachingAssistant → Student + Staff	Multiple inheritance	A teaching assistant has both roles.
ResearchScholar → Student	Multilevel inheritance	A research scholar is a specialised student.
Address/Course as data members	Composition (has-a)	A person has an address; a student has a course.

Task 2.2 – Resolve the Diamond

Without virtual inheritance, TeachingAssistant receives one Person subobject through Student and another Person subobject through Staff. This is the diamond problem. Accessing a common member such as name through a TeachingAssistant object becomes ambiguous because the compiler cannot choose which Person subobject is intended. Declaring Student and Staff as virtual public bases of Person makes the most-derived TeachingAssistant contain only one shared Person subobject. The shared virtual base is initialized by the most-derived class.

Task 2.3 – Design the Classes

```
#include <iostream>
#include <string>

class Person {
protected:
    std::string name;
    int id;
```

```
    std::string phone;
public:
    Person(const std::string& n, int i, const std::string& p)
        : name(n), id(i), phone(p) {}
};

class Student : virtual public Person {
protected:
    std::string programme;
    double cgpa;
public:
    Student(const std::string& n, int i, const std::string& p,
            const std::string& prog, double g)
        : Person(n, i, p), programme(prog), cgpa(g) {}
};

class Staff : virtual public Person {
protected:
    std::string department;
    double salary;
public:
    Staff(const std::string& n, int i, const std::string& p,
          const std::string& dept, double s)
        : Person(n, i, p), department(dept), salary(s) {}
};

class TeachingAssistant : public Student, public Staff {
public:
    TeachingAssistant(const std::string& n, int i, const std::string& p,
                      const std::string& prog, double g,
                      const std::string& dept, double s)
        : Person(n, i, p),
```

```
        Student(n, i, p, prog, g),
        Staff(n, i, p, dept, s) {}
};

class ResearchScholar : public Student {
    double fellowship;
public:
    ResearchScholar(const std::string& n, int i, const std::string& p,
                    const std::string& prog, double g, double f)
        : Person(n, i, p), Student(n, i, p, prog, g), fellowship(f) {}
};
```

The Person, Student and Staff relationships are public because the derived objects should be usable wherever their base type is expected. TeachingAssistant publicly inherits Student and Staff so both role interfaces remain accessible. Virtual inheritance is used only for the shared Person base.

Task 2.4 – Access Table

Person member	**Student : public Person**	**Student : protected Person**	**Student : private Person**
public	public	protected	private
protected	protected	protected	private
private	Not directly accessible	Not directly accessible	Not directly accessible

A private member of Person is not directly accessible inside Student under any inheritance mode. It can be accessed only through suitable public or protected member functions provided by Person.

Predict the Output – Virtual Base Classes

Prediction written before execution:

Person Student Staff TA

Expected program output:

Person Student Staff TA

Person is printed once because Student and Staff virtually inherit Person. In a TeachingAssistant object, the most-derived class creates and initializes one shared Person virtual base before constructing its direct base classes.

If virtual is removed from both declarations, the predicted output becomes:

Person Student Person Staff TA

There is no compiler error merely from constructing the object and printing these constructor messages. However, an expression such as ta.name would be ambiguous because TeachingAssistant contains two Person subobjects. The compiler reports an ambiguous member access if such a name is used.

Alternative Concept and Complexity Analysis

If TeachingAssistant contains Student and Staff objects as members, the relationship becomes has-a rather than is-a. The class would contain Student student; and Staff staff;. Each contained object has its own Person base, so there are two Person subobjects. The object is larger than a design with one shared virtual Person base in most implementations; sizeof should be measured because padding and compiler ABI affect exact sizes. Access becomes explicit, for example ta.student.getName() or ta.staff.getName(), so the ambiguity is removed but the object no longer behaves as a Student or Staff through inheritance. Accessing a field is O(1) in either design.

TASK 3 – CONSTRUCTORS IN DERIVED CLASSES AND MEMBER CLASSES

Task 3.1 – Pass Values to Base Constructors

The derived constructor uses an initializer list to pass values to its base classes. When Person is a virtual base, the most-derived class (TeachingAssistant) must initialize Person directly. If a Student or Staff object is created on its own, that class initializes Person. When TeachingAssistant is created, its Student and Staff initializers for the virtual Person are ignored; the TeachingAssistant initializer is used.

```
#include <iostream>
#include <string>

class Person {
protected:
    std::string name;
    int id;
public:
```

```
    Person(const std::string& n, int i) : name(n), id(i) {
        std::cout << "Person constructed\n";
    }
    virtual ~Person() { std::cout << "Person destroyed\n"; }
};

class Student : virtual public Person {
protected:
    std::string programme;
public:
    Student(const std::string& n, int i, const std::string& p)
        : Person(n, i), programme(p) {
        std::cout << "Student constructed\n";
    }
    ~Student() override { std::cout << "Student destroyed\n"; }
};

class Staff : virtual public Person {
protected:
    std::string department;
public:
    Staff(const std::string& n, int i, const std::string& d)
        : Person(n, i), department(d) {
        std::cout << "Staff constructed\n";
    }
    ~Staff() override { std::cout << "Staff destroyed\n"; }
};

class TeachingAssistant : public Student, public Staff {
public:
    TeachingAssistant(const std::string& n, int i,
                      const std::string& p, const std::string& d)
```

```
        : Person(n, i), Student(n, i, p), Staff(n, i, d) {
        std::cout << "TeachingAssistant constructed\n";
    }
    ~TeachingAssistant() override {
        std::cout << "TeachingAssistant destroyed\n";
    }
};
```

Task 3.2 – Member Classes

Address is a member of Person (has-a), and Course is a member of Student (has-a). Their constructors are called through the containing class initializer list.

```
class Address {
    std::string city;
    int pin;
public:
    Address(const std::string& c, int p) : city(c), pin(p) {}
    std::string str() const {
        return city + "-" + std::to_string(pin);
    }
};

class Course {
    std::string code;
public:
    explicit Course(const std::string& c) : code(c) {}
    std::string getCode() const { return code; }
};

// Example member declarations:
// Person has: Address home;
// Student has: Course course;
// Initializer-list example:
// Person(..., const Address& a) : name(...), id(...), home(a) {}
```

```
// Student(..., const Course& c) : Person(...), course(c) {}
```

Task 3.3 – Trace the Order

For the supplied Vehicle/Engine/Car program, the base class is constructed first, then member objects in declaration order, then the derived class body. Destruction happens in reverse: derived destructor, members in reverse declaration order, then base destructor.

Prediction written before execution:

```
Vehicle Engine Car | ~Car ~Engine ~Vehicle
```

Expected output:

```
Vehicle Engine Car | ~Car ~Engine ~Vehicle
```

Task 3.4 – Experiment

When virtual is removed from Student and Staff, TeachingAssistant contains two Person subobjects instead of one. Person's constructor runs twice for a TeachingAssistant object. sizeof(TeachingAssistant) generally increases by at least the storage needed for the additional Person subobject, but the exact byte count is compiler- and ABI-dependent. Record the actual sizeof values from the compiler used.

Complete the Code – TODO 1

Completed constructor using a member initializer list:

```
#include <iostream>
#include <string>

class Person {
protected:
    std::string name;
    int age;
public:
    Person(const std::string& n, int a) : name(n), age(a) {}
};

class Address {
    std::string city;
    int pin;
public:
```

```
    Address(const std::string& c, int p) : city(c), pin(p) {}
    std::string str() const {
        return city + "-" + std::to_string(pin);
    }
};

class Student : public Person {
    long long regNo;
    Address home;
public:
    Student(const std::string& n, int a, long long r,
            const std::string& city, int pin)
        : Person(n, a), regNo(r), home(city, pin) {}

    void print() const {
        std::cout << regNo << ' ' << name << " (" << age << ") "
                  << home.str() << '\n';
    }
};

int main() {
    Student s("Kavya", 19, 192311001, "Salem", 636001);
    s.print();
}
```

Expected output:

```
192311001 Kavya (19) Salem-636001
```

If the initializer list is written as home(city, pin), regNo(r), Person(n, a), the actual initialization order does not change. C++ initializes the base class first and then data members in the order they are declared in the class: regNo before home. With -Wall -Wextra, the compiler can issue a -Wreorder warning because the written initializer-list order differs from the declaration order.

Alternative Concept and Complexity Analysis

If home were an Address* allocated with new, the constructor would allocate memory and the class would need a destructor to delete it. A correct copy constructor and copy-assignment operator would also be needed to perform a deep copy; otherwise two Student objects could point to the same Address and cause double deletion or dangling pointers. A pointer adds pointer storage and dynamic allocation has runtime and allocator overhead. A direct Address member is simpler, automatically constructed and destroyed, and is preferred when the address is always part of a Student.

TASK 4 – POINTERS TO DERIVED CLASSES: FORENSICS

Task 4.1 – Locate the Defects

- rate() is not virtual in Account. Calling it through Account* uses static binding, so a SeniorAccount pointer viewed as Account* returns 3.5 instead of 4.0.
- Account has a non-virtual destructor. Deleting a SeniorAccount object through Account* has undefined behaviour; the derived destructor is not reliably called.
- SeniorAccount allocates history with new[] and releases it only in its destructor. If the derived destructor is skipped, history leaks.
- Account copy = SeniorAccount() performs object slicing. Only the Account base portion is copied, so the derived SeniorAccount part is discarded.

Task 4.2 – Construct Evidence

Add diagnostic messages to both destructors and compile with AddressSanitizer. Example command:

```
g++ -std=c++17 -Wall -Wextra -fsanitize=address -fno-omit-frame-pointer task4_bug.cpp -o task4_bug
```

The defective delete-through-base example has undefined behaviour because the base destructor is not virtual. Depending on the compiler and sanitizer runtime, the run may report a new-delete-type-mismatch, a leak, or another symptom. Do not copy a guessed sanitizer transcript as actual evidence. Run it on your machine and insert the real terminal screenshot and report. A reliable demonstration should compare the destructor messages and sanitizer output before and after the repair.

Task 4.3 – Repair the Program

```
#include <iostream>
#include <memory>
#include <vector>
```

```
#include <array>

class Account {
public:
    virtual double rate() const { return 3.5; }
    virtual ~Account() {
        std::cout << "Account destroyed\n";
    }
};

class SeniorAccount : public Account {
    std::array<double, 100> history{};
public:
    double rate() const override { return 4.0; }
    ~SeniorAccount() override {
        std::cout << "SeniorAccount destroyed\n";
    }
};

int main() {
    std::vector<std::unique_ptr<Account>> accounts;
    accounts.push_back(std::make_unique<Account>());
    accounts.push_back(std::make_unique<SeniorAccount>());

    for (const auto& account : accounts)
        std::cout << account->rate() << '\n';
}
```

Expected rate output:

```
3.5
4
```

The virtual rate() function selects the derived override at run time. The virtual destructor ensures the correct derived destructor is called. unique_ptr owns each object and releases it automatically. std::array avoids a manual new[]/delete[] pair.

Task 4.4 – Prevent Recurrence

- If a base class is used polymorphically, give it a virtual destructor.
- Use override on every intended virtual override so the compiler checks the function signature.
- Prefer RAII containers and smart pointers such as std::array, std::vector and std::unique_ptr instead of owning raw pointers.
- Store polymorphic objects through pointers or references; do not copy derived objects into base objects when the derived state must be preserved.

Predict the Output – Static and Dynamic Binding

Prediction, assuming the intended separators are single spaces:

3.5 4 3.5

Explanation: p->rate() calls the non-virtual Account::rate() because the function is not virtual; the call is bound at compile time and returns 3.5. p->vrate() calls the virtual function and dispatches to SeniorAccount::vrate() at run time, returning 4.0. Account copy = s slices the object and creates only an Account object, so copy.vrate() returns 3.5.

Source-format note: the assignment text appears to show a multi-character character literal in the output separators. A multi-character character literal is not a string separator and can produce a compiler warning and implementation-dependent integer value. Use " " or a single-character ' ' for readable spaces. The predicted values above refer to the intended three values.

Alternative Concept and Complexity Analysis

A vector of std::unique_ptr<Account> stores owning pointers to polymorphic objects. Processing n accounts remains O(n), and each rate() call is O(1). The pointer-based design has a small pointer/control-block or allocation overhead, but avoids copying and slicing. The virtual-destructor defect is prevented by the virtual base destructor. Memory leaks from manual ownership are reduced because unique_ptr manages lifetime. Object slicing is avoided because the vector stores pointers, not Account values. A non-virtual rate() function would still cause static binding, so it must remain virtual.

TASK 5 – POINTERS TO OBJECTS AND THE this POINTER

Task 5.1 – Four Ways to Reach an Object

```
#include <iostream>
#include <string>
#include <iomanip>

class Cart {
    std::string items[10];
    double prices[10];
    int n = 0;
    double discount = 0.0;
public:
    Cart& add(const std::string& item, double price) {
        if (n < 10) {
            items[n] = item;
            prices[n] = price;
            ++n;
        }
        return *this;
    }

    Cart& setDiscount(double discount) {
        this->discount = discount;
        return *this;
    }

    double total() const {
        double sum = 0.0;
        for (int i = 0; i < n; ++i)
            sum += prices[i];
        return sum * (100.0 - discount) / 100.0;
    }
};
```

```
int main() {
    Cart direct;                    // (a) direct object
    direct.add("Book", 100);

    Cart* pointer = &direct;        // (b) pointer to object
    pointer->add("Pen", 20);

    Cart& reference = direct;       // (c) reference
    reference.setDiscount(10);

    Cart* dynamicCart = new Cart;   // (d) dynamic object
    dynamicCart->add("Bag", 500);
    std::cout << direct.total() << '\n';
    std::cout << dynamicCart->total() << '\n';
    delete dynamicCart;
}
```

The direct object is accessed by its variable name. A pointer uses -> to access members. A reference is an alias for the existing object. A dynamically allocated object is accessed through a pointer and must be released with delete. The this pointer points to the current object inside a non-static member function. *this means the current object itself.

Task 5.2 – Highest Total in an Array of Cart Pointers

```
Cart c1, c2, c3;
c1.add("Rice", 60);
c2.add("Laptop", 500);
c3.add("Oil", 180);

Cart* carts[] = { &c1, &c2, &c3 };
int count = 3;
int best = 0;

for (int i = 1; i < count; ++i) {
    if (carts[i]->total() > carts[best]->total())
```

```
        best = i;
    }

    std::cout << "Highest cart total: "
              << carts[best]->total() << '\n';
```

Task 5.3 – Compare the Four Ways

Access method	Lifetime / safety	Typical use
Direct object	Automatic lifetime; safe and simple	Normal local objects
Pointer to existing object	Does not own object; can be null or dangling	Optional access or polymorphism
Reference	Alias; cannot normally be reseated; must refer to a valid object	Function parameters and aliases
new/delete pointer	Dynamic lifetime; programmer must manage ownership	When lifetime must outlive the current scope; prefer smart pointers

Complete the Code – TODO 1 and TODO 2

The completed add() method stores the item and price only when the fixed array has space and returns *this by reference. setDiscount() uses this->discount to distinguish the member from the parameter.

```
#include <iomanip>
#include <iostream>
#include <string>

class Cart {
    std::string items[10];
    double prices[10];
    int n = 0;
    double discount = 0.0;
public:
```

```
    Cart& add(const std::string& item, double price) {
        if (n < 10) {
            items[n] = item;
            prices[n] = price;
            ++n;
        }
        return *this;
    }

    Cart& setDiscount(double discount) {
        this->discount = discount;
        return *this;
    }

    double total() const {
        double t = 0;
        for (int i = 0; i < n; ++i)
            t += prices[i];
        return t * (100 - discount) / 100;
    }
};

int main() {
    Cart c;
    c.add("Rice", 60).add("Dal", 120).setDiscount(10);
    Cart* p = &c;
    p->add("Oil", 180);
    std::cout << std::fixed << std::setprecision(2)
              << p->total() << '\n';
}
```

Expected output:

```
324.00
```

If add() returned Cart by value, each chained call would return a copy. This can cause extra copying and subsequent calls would operate on temporary copies rather than the original object. Returning Cart& returns the same object and supports efficient chaining.

Alternative Concept and Complexity Analysis

With std::vector<Cart>, the program stores Cart objects directly instead of storing one pointer per cart. Finding the highest total remains O(n), because every cart must be checked. A vector of objects stores the objects themselves and may move or copy them when it grows. A vector of pointers stores pointer values, while the Cart objects live elsewhere. Pointers, references and iterators into a vector can be invalidated when the vector reallocates; pointers to separately allocated objects owned by smart pointers remain valid when the vector of smart pointers reallocates.

TASK 6 – DESIGN CHALLENGE: VEHICLE RENTAL FLEET

Stage 1 – Interpret the Problem

Entities: Vehicle, Bicycle, MotorVehicle, TwoWheeler, Car, ElectricCar and ElectricBus. Common data are registration number and daily rate. Motor vehicles also have an energy level and service due date. Cars have seating capacity. Electric cars have battery capacity and a per-kWh charge. Common behaviours are rent, return and display. The charge calculation differs by vehicle type.

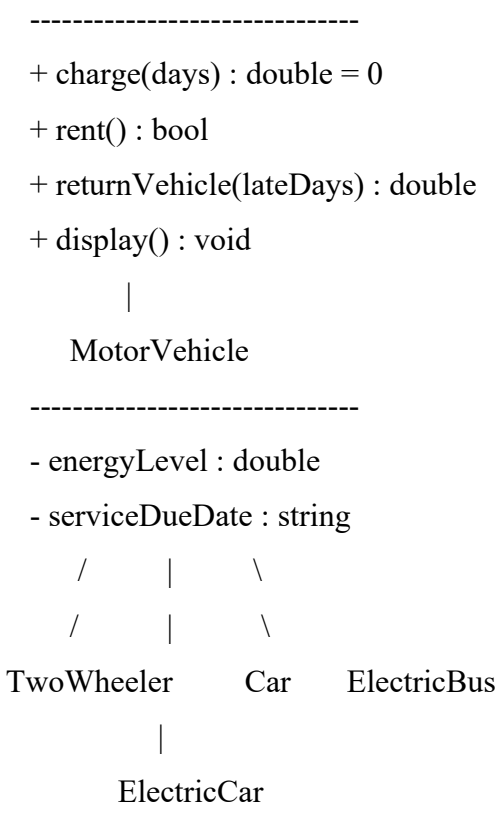
Testable requirements:

- Every vehicle must have a non-empty registration number and a non-negative daily rate.
- A vehicle that is already rented cannot be rented again.
- A successful rental calculates a charge using the vehicle's own charge() implementation.
- Returning a vehicle marks it available and adds a late fee when it is returned after the due date.
- The fleet can list every vehicle through a common base-class pointer.
- A new derived vehicle type can be added without editing existing vehicle classes.

Stage 2 – Propose the Hierarchy

Vehicle (abstract)

- registrationNo : string
- dailyRate : double
- rented : bool



Bicycle also derives directly from Vehicle.

Inheritance types: Bicycle uses single inheritance from Vehicle. MotorVehicle uses single inheritance from Vehicle. TwoWheeler, Car and ElectricBus use hierarchical inheritance from MotorVehicle. ElectricCar uses multilevel inheritance through Car and MotorVehicle. Registration and rate information are common base-class data; a future Address or ServiceRecord could be modelled as a member class if the requirement expands.

Stage 3 – Critical Design Decisions

- Vehicle is abstract because every concrete vehicle must supply its own charge(days) implementation.
- charge() and the destructor are virtual. rent(), returnVehicle() and display() are common operations.
- The fleet is held as a collection of owning base-class pointers: std::vector<std::unique_ptr<Vehicle>>. This is a dynamically sized array-like container and safely owns polymorphic objects.
- MotorVehicle stores common energy/service information. Car stores seating capacity. ElectricCar stores battery capacity and a per-kWh charge rate.

- Late return charges are calculated by the common return operation, so the rule is not duplicated in every derived class.

Stage 4 – Class Declarations and Pseudocode

SIMATS-style algorithmic pseudocode – Rent Vehicle:

- START
- INPUT the registration number to rent
- SEARCH the fleet for a vehicle with that registration number
- IF no vehicle is found, DISPLAY 'Vehicle not found' and STOP
- IF the vehicle is already rented, DISPLAY 'Already rented' and STOP
- CALL rent() on the selected vehicle
- INPUT the number of rental days
- CALCULATE charge by calling the selected object's virtual charge(days)
- DISPLAY the rental charge
- STOP

SIMATS-style algorithmic pseudocode – Return Vehicle:

- START
- INPUT the registration number and late days
- SEARCH the fleet for the matching vehicle
- IF no vehicle is found, DISPLAY 'Vehicle not found' and STOP
- IF the vehicle is not rented, DISPLAY 'Vehicle was not rented' and STOP
- CALL returnVehicle(lateDays)
- CALCULATE late fee and mark the vehicle as available
- DISPLAY the late fee and return status
- STOP

Stage 5 – Implementation and Demonstration

```
#include <iostream>
#include <iomanip>
#include <memory>
#include <string>
#include <vector>

class Vehicle {
```

```
protected:
    std::string registrationNo;
    double dailyRate;
    bool rented = false;

public:
    Vehicle(const std::string& reg, double rate)
        : registrationNo(reg), dailyRate(rate) {}

    virtual double charge(int days) const = 0;

    bool rent() {
        if (rented) return false;
        rented = true;
        return true;
    }

    double returnVehicle(int lateDays) {
        if (!rented) return -1.0;
        const double lateFee = lateDays > 0 ? lateDays * 200.0 : 0.0;
        rented = false;
        return lateFee;
    }

    virtual void display() const {
        std::cout << registrationNo << " | Daily rate: "
                  << dailyRate << " | "
                  << (rented ? "Rented" : "Available") << '\n';
    }

    virtual ~Vehicle() = default;
};
```

```
class Bicycle : public Vehicle {
public:
   Bicycle(const std::string& reg, double rate)
      : Vehicle(reg, rate) {}
   double charge(int days) const override {
      return dailyRate * days;
   }
};

class MotorVehicle : public Vehicle {
protected:
   double energyLevel;
   std::string serviceDueDate;
public:
   MotorVehicle(const std::string& reg, double rate,
                double energy, const std::string& due)
      : Vehicle(reg, rate), energyLevel(energy),
        serviceDueDate(due) {}
};

class TwoWheeler : public MotorVehicle {
public:
   TwoWheeler(const std::string& reg, double rate,
              double fuel, const std::string& due)
      : MotorVehicle(reg, rate, fuel, due) {}
   double charge(int days) const override {
      return dailyRate * days + 50.0 * days;
   }
};

class Car : public MotorVehicle {
```

```
protected:
    int seatingCapacity;
public:
    Car(const std::string& reg, double rate, double fuel,
        const std::string& due, int seats)
        : MotorVehicle(reg, rate, fuel, due),
          seatingCapacity(seats) {}
    double charge(int days) const override {
        return dailyRate * days + 100.0 * days;
    }
};

class ElectricCar : public Car {
    double batteryCapacity;
    double chargePerKWh;
public:
    ElectricCar(const std::string& reg, double rate,
                double battery, double perKWh,
                const std::string& due, int seats)
        : Car(reg, rate, 0.0, due, seats),
          batteryCapacity(battery), chargePerKWh(perKWh) {}
    double charge(int days) const override {
        return dailyRate * days
            + batteryCapacity * chargePerKWh * days;
    }
};

// New type added without modifying the existing classes.
class ElectricBus : public MotorVehicle {
    int passengerCapacity;
    double batteryCapacity;
    double chargePerKWh;
```

```
public:
    ElectricBus(const std::string& reg, double rate,
                double battery, double perKWh,
                const std::string& due, int passengers)
        : MotorVehicle(reg, rate, battery, due),
          passengerCapacity(passengers),
          batteryCapacity(battery), chargePerKWh(perKWh) {}
    double charge(int days) const override {
        return dailyRate * days
            + batteryCapacity * chargePerKWh * days;
    }
};

int main() {
    std::vector<std::unique_ptr<Vehicle>> fleet;
    fleet.push_back(std::make_unique<Bicycle>("BI-101", 100));
    fleet.push_back(std::make_unique<TwoWheeler>(
        "TW-202", 300, 8.0, "2026-12-01"));
    fleet.push_back(std::make_unique<Car>(
        "CA-303", 1500, 40.0, "2026-12-10", 5));
    fleet.push_back(std::make_unique<ElectricCar>(
        "EV-404", 1800, 50.0, 8.0, "2026-12-15", 5));
    fleet.push_back(std::make_unique<ElectricBus>(
        "EB-505", 5000, 120.0, 8.0, "2026-12-20", 40));

    std::cout << std::fixed << std::setprecision(2);

    for (const auto& v : fleet) {
        v->display();
        if (v->rent()) {
            std::cout << "Rental charge for 2 days: "
                      << v->charge(2) << '\n';
```

```
            double lateFee = v->returnVehicle(1);
            std::cout << "Late fee for 1 day: "
                  << lateFee << "\n\n";
        }
    }
}
```

Illustrative expected calculations for two rental days and one late day (the exact display also includes vehicle status lines):

Vehicle	**Charge calculation**	**Rental charge**
Bicycle	100 × 2	200.00
TwoWheeler	(300 + 50) × 2	700.00
Car	(1500 + 100) × 2	3200.00
ElectricCar	(1800 + 50 × 8) × 2	4400.00
ElectricBus	(5000 + 120 × 8) × 2	11920.00

The one-day late fee is 200.00 for each returned vehicle. The energy and service fields are retained as vehicle information; the sample charge formulas are illustrative assignment rules and can be changed if the faculty provides a specific fuel or electricity pricing formula.

Alternative Concept and Complexity Analysis

A single Vehicle class with a type code and a switch in charge() can calculate all charges. Both a switch with a fixed number of cases and a virtual function call take O(1) time per charge calculation. For n vehicles, processing the fleet takes O(n). The single-class design stores fields for all vehicle types, so many objects carry unused data. A polymorphic object may contain a vtable pointer in common implementations, but each derived object stores only its relevant data. Adding a vehicle type to the single-class design requires changing the type code, switch statement and often constructor/display logic. With inheritance, a new derived class can implement charge() without changing existing vehicle classes.